

I've gone through a meaningful slice of your build system (not just skimming filenames), and I'm going to give you a **straight, systems-level assessment** of what you improved—and where the architecture is still fragile.

High-confidence assessment

You've made **real, substantive improvements**. This is not superficial cleanup.

Specifically, you've moved toward:

- **Explicit environment construction**
- **Per-package staging isolation**
- **A quasi-sysroot model for multi-pass builds**

That aligns directly with the failure modes you were hitting earlier.

However—and this is important—you are still in a **“hybrid model”** that will eventually bite you again under more complex packages (glib/GI was just the first stress test).

What you did *right* (and it matters)

1. Staging-as-sysroot (this is the big one)

Your logic here:

```
env["PKG_CONFIG_PATH"] = f"{staging}/usr/lib/pkgconfig:..."
env["LD_LIBRARY_PATH"] = f"{staging}/usr/lib:..."
env["PATH"] = f"{staging}/usr/bin:..."
```

This is **exactly the correct direction**.

You've effectively created a **progressive sysroot overlay**, which:

- Allows multi-pass builds to “see” prior artifacts
- Avoids polluting the host
- Keeps dependency resolution localized

👉 This directly addresses your earlier:

- glib header discovery failure
- pkg-config mismatch
- missing staged artifacts

Verdict: structurally correct

2. DESTDIR discipline (mostly correct now)

You fixed a very common but subtle bug:

```
env.pop("DESTDIR", None) # instead of empty string
```

This is *not trivial*—many build systems interpret:

- DESTDIR="" → still enabled
- unset → disabled

👉 This likely fixed silent mis-installs.

3. Staging filesystem normalization

This block:

```
for d in ("usr/bin", "usr/lib", "usr/sbin"):
    ...
for link in ("bin", "lib", "sbin"):
    os.symlink(f"usr/{link}", ...)
```

You're aligning staging layout with:

- modern /usr-merged systems
- BLFS expectations

This eliminates class-of-path bugs, not just individual ones.

4. Build execution model (clean and auditable)

Your run_command:

- streams output (no buffering)
- captures stderr/stdout
- logs everything

This is **exactly what you want** for debugging build systems.

Where things are still fragile (this is the important part)

! 1. You are not a *true sysroot* yet

Right now you are simulating a sysroot via environment variables.

That works for:

- many autotools projects
- simple meson builds

But it breaks for:

- **g-ir-scanner (GI)**
- some meson/cmake configurations

- anything that probes /usr directly

What's missing:

You are NOT setting:

--sysroot

or equivalent compiler flags:

CFLAGS="--sysroot=..."

LDFLAGS="--sysroot=..."

Why this matters:

GI and some build systems:

- ignore PKG_CONFIG_PATH
- directly invoke compiler probes
- resolve against /usr/include implicitly

👉 Your current system still allows **host leakage**

! 2. PKG_CONFIG_PATH is incomplete (subtle bug)

You set:

```
env["PKG_CONFIG_PATH"] = "/usr/lib/pkgconfig:/usr/share/pkgconfig"
```

Then prepend staging paths.

Problem:

You are still including:

/usr/lib/pkgconfig

That means:

- host .pc files can override staged ones
- you get **non-deterministic builds**

Correct model:

You want:

PKG_CONFIG_LIBDIR=<staging-only>

PKG_CONFIG_PATH=<optional overlays>

👉 PKG_CONFIG_LIBDIR **replaces**, not augments

! 3. LD_LIBRARY_PATH is doing too much heavy lifting

You rely on:

LD_LIBRARY_PATH = staging libs

This works at runtime, but:

- linkers don't always respect it fully
- some configure scripts bypass it
- it can mask missing rpaths

Proper approach:

You should also inject:

LDFLAGS="-Wl,-rpath,<staging>/usr/lib"

or better:

- enforce correct linking at build time
- not runtime hacks

! 4. GI-specific gap (this WILL come back)

I did not see:

GI_TYPELIB_PATH

or explicit handling for:

- .gir scanning
- typelib resolution

Why this matters:

g-ir-scanner:

- executes compiled binaries
- inspects them dynamically
- needs both:
 - headers
 - runtime libs
 - typelib paths

👉 Your current setup will likely:

- work for glib now
- break later for GTK / GStreamer / etc.

! 5. PATH injection order risk

```
env["PATH"] = f"{staging}/usr/bin:{staging}/usr/sbin:" + env["PATH"]
```

This is good—but incomplete.

Missing:

- isolation from host /usr/bin
- potential toolchain contamination

Long-term issue:

You're not distinguishing between:

- **build tools**
- **target tools**

This becomes critical with:

- cross-compilation
- bootstrap phases

! 6. No hard isolation boundary

Right now:

- environment variables *suggest* isolation
- but nothing enforces it

What's missing:

- chroot or namespace enforcement at build step level
- or strict sysroot enforcement

You *have* chroot scripts—but your Python executor isn't inherently tied to that isolation.

Architectural diagnosis

You are currently in:

“Environment-driven pseudo-sysroot model”

This is:

-  flexible
-  easy to debug
-  not fully deterministic
-  prone to edge-case breakage

What I would do next (priority order)

1. Introduce real sysroot semantics

Add:

```
CFLAGS="--sysroot=$staging"  
LDFLAGS="--sysroot=$staging"
```

And propagate through all builds.

2. Replace `PKG_CONFIG_PATH` with:

```
PKG_CONFIG_LIBDIR=$staging/usr/lib/pkgconfig:$staging/usr/lib64/pkgconfig
```

Remove `/usr/lib/pkgconfig` entirely.

3. Add GI support explicitly

GI_TYPELIB_PATH=\$staging/usr/lib/girepository-1.0

And ensure:

- scanner can execute against staged libs

4. Gradually eliminate host fallback

Audit for:

- /usr/include
- /usr/lib
- implicit compiler defaults

5. Decide your model (important fork)

You need to choose:

Option A — Full sysroot model (recommended)

- deterministic
- reproducible
- scalable

Option B — Environment overlay model (current)

- simpler
- but fragile at scale

Right now you are **mid-transition**

Bottom line

You fixed:

- the *symptoms* (correctly)
- and partially the *underlying model*

But:

You have not yet:

- fully eliminated host dependency leakage
- formalized sysroot semantics
- hardened GI behavior

Final verdict

This is a strong iteration.

You're now operating at a level where:

- failures will be rarer
- but more subtle and harder to debug

Which is exactly where sysroot correctness becomes mandatory.

If you want, next step I can:

- trace one full package (e.g. glib) through your pipeline
- and show exactly where it's still relying on host assumptions

That would give you a concrete before/after validation of the model.

